

Marketplace Builder Hackathon 2025

Prepared By: Asif Ali

Marketplace Technical Foundation Presentation

Introduction

This document outlines the technical foundation for your food business marketplace. It focuses on defining the system architecture, workflows, API requirements, and Sanity CMS schema to ensure seamless integration between frontend and backend components for efficient and scalable operations.

1. Technical Requirements

Frontend Requirements

- **Responsive Design:** Ensure a seamless user experience on both mobile and desktop devices.
- **Essential Pages:**
 - Home
 - Menu
 - Blog
 - About
 - Shop
 - Product Details (/products/{id} - dynamic page)
 - Sign In
 - Sign Up
 - Contact
 - Checkout
 - Order Confirmation

Backend Requirements (Sanity CMS)

- Utilize **Sanity CMS** for efficient data management:
 - Product Information
 - Customer Details
 - Order Records
- Define structured schemas to align with:
 - **Product Structure** (e.g., name, price, stock)
 - **Order Structure** (e.g., customer info, product details, payment status)

Third-Party APIs

- **Shipment Tracking API:** Fetch and display real-time order shipment status.
 - **Payment Gateway API:** Securely process payments and return confirmation statuses.
-

2. System Architecture

High-Level Architecture Diagram

```
[Frontend (Next.js)]
|
[Sanity CMS] <-----> [Product Data API]
|
[Third-Party APIs] ---> [Shipment Tracking API]
|
|---> [Payment Gateway]
```

Workflow Description

1. Product Browsing:

- The user accesses the marketplace frontend (built with Next.js).
- The frontend requests product data via the Sanity CMS API.
- Products are dynamically displayed.

2. Order Placement:

- The user adds products to the cart and proceeds to checkout.
- Order details are stored in Sanity CMS.
- Payment is securely processed via the Payment Gateway.

3. Shipment Tracking:

- The system updates the order status using the Shipment Tracking API.
 - Real-time tracking information is displayed to the customer.
-

3. Sanity CMS Schemas

Product Schema

```
export default {
  name: 'product',
  type: 'document',
```

```

    fields: [
      { name: 'name', type: 'string', title: 'Product Name' },
      { name: 'price', type: 'number', title: 'Price' },
      { name: 'stock', type: 'number', title: 'Stock Level' },
      { name: 'image', type: 'image', title: 'Product Image' }
    ]
  };

```

Order Schema

```

export default {
  name: 'order',
  type: 'document',
  fields: [
    { name: 'customer', type: 'reference', to: [{ type: 'customer' }], title:
'Customer' },
    { name: 'products', type: 'array', of: [{ type: 'reference', to: [{ type:
'product' }]}], title: 'Products' },
    { name: 'totalAmount', type: 'number', title: 'Total Amount' },
    { name: 'status', type: 'string', title: 'Order Status' }
  ]
};

```

4. Workflow Diagram

